

FAST-MEL: A Fast, Accurate, and Storage Efficient Solution for Multimodal Entity Linking

Thomas Derrien
thomas.derrien@irisa.fr
Univ. Rennes, INSA Rennes, CNRS,
Inria, IRISA - UMR 6074
Rennes, France

Laurent Amsaleg
laurent.amsaleg@irisa.fr
Univ. Rennes, CNRS, Inria, IRISA -
UMR 6074
Rennes, France

Pascale Sébillot
pascale.sebillot@irisa.fr
Univ. Rennes, INSA Rennes, CNRS,
Inria, IRISA - UMR 6074
Rennes, France

Abstract

Multimodal entity linking (MEL) is the task that consists of matching textual and visual mentions of entities in unstructured data to their corresponding entities in a knowledge base (KB). To be effective in large-scale practical settings, MEL systems must meet three objectives: high linking accuracy, computational efficiency, and storage efficiency, i.e., a compact yet efficient index of the KB. In this paper, we highlight that state-of-the-art systems fail to simultaneously satisfy these 3 requirements. To meet this three-fold objective, we propose FAST-MEL, a lightweight encoder-based MEL solution that relies on a novel and compact fixed-size vectorized representation of both the textual and visual information of each entity or mention. It matches the accuracy of the best systems but performs three orders of magnitude faster. It also consumes one order of magnitude less storage than the fastest systems.

CCS Concepts

• Information systems → Multimedia databases; • General and reference → Performance.

Keywords

Multimodal Entity Linking, Accuracy, Computational Efficiency, Storage Efficiency, Knowledge Base

ACM Reference Format:

Thomas Derrien, Laurent Amsaleg, and Pascale Sébillot. 2026. FAST-MEL: A Fast, Accurate, and Storage Efficient Solution for Multimodal Entity Linking. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*, July 20–24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3805712.3809860>

1 Introduction

Entity Linking (EL) consists of grounding entity *mentions* in a document to their corresponding *entities* in a knowledge base (KB). It can be naturally formulated as a retrieval problem over a KB, where a possibly ambiguous mention in a sentence acts as a query and the goal is to retrieve the correct entity in the KB. EL is a core component of many downstream applications such as information retrieval and question answering [6, 9, 19]. Recently, Multimodal Entity Linking (MEL) [10] has been introduced to leverage visual

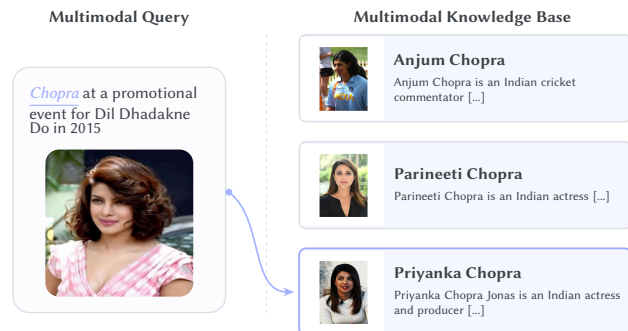


Figure 1: Example of MEL from WikiMEL.

context alongside text in order to reduce ambiguity and improve linking accuracy. A typical MEL setup is illustrated in Figure 1.

In this setting, a multimodal query comprises a text that includes an entity mention and an associated image, while each entity in the KB is represented by a brief textual description along with an illustrative image.

Most MEL approaches [1, 2, 7, 8, 15, 20] adopt a representation-based paradigm, in which multimodal queries and entities are projected into a shared embedding space using CLIP [11], and are linked via similarity-based matching. These encoder-based models are usually computationally lightweight and perform fast inference. However, these methods generally fall short in accuracy and require, for each entity in the KB, storing a feature vector for every textual token in its description and for every image patch in its associated image, which leads to a storage-intensive index. More recently, Large Language Models (LLMs) and Multimodal-Large-Language Models (MLLMs) have been applied to MEL. They serve several purposes: enriching the textual part of the query with synthetic context [3, 4], reformulating entity descriptions [4], reranking a set of pre-selected candidate entities [2, 3, 5, 7], or directly generating the corresponding entity name [14]. These models achieve state-of-the-art accuracy by leveraging massive pretraining and reasoning capabilities, but come with higher computational costs due to large parameter counts and autoregressive inference. They may also introduce dependencies on external APIs for closed source models. While their storage footprint can be moderate, the overall cost in terms of computing and deployment resources is not negligible.

In large-scale practical settings, MEL systems must balance three requirements: (1) *storage efficiency*: KBs can contain millions of entities (e.g., more than 6M entities of type Human in Wikidata), making compact indexing essential; (2) *computational efficiency*:



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGIR '26, Melbourne, VIC, Australia*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2599-9/2026/07

<https://doi.org/10.1145/3805712.3809860>

Inference must be as fast as possible; (3) *high accuracy*: Entity linking must be as accurate as possible.

Despite its importance, this triple trade-off hasn't been studied in prior work. In this paper, we study MEL methods from this perspective. We first highlight that existing approaches fail to satisfy all three requirements simultaneously. To meet this three-fold objective, we propose FAST-MEL¹, an encoder-based architecture built upon CLIP. Our approach relies on a novel, compact, fixed-size representation that fuses token- and patch-level information into a single 512-dimensional vector per query or entity. As a result, it requires one order of magnitude less storage than the fastest existing solutions (i.e., encoder-based systems). FAST-MEL also matches the accuracy of the best-performing systems (i.e., LLM/MLLM-based approaches) while running three orders of magnitude faster. By providing a lightweight, accurate and storage-efficient solution, our approach offers a practical alternative for large-scale MEL systems where deployment costs are critical.

2 Task Definition and Analysis of Existing Systems

The MEL task is formulated as follows. Let $q = \{q_m, q_t, q_v\}$ a multi-modal query where q_m is the textual mention of an entity, q_t is its textual context (i.e., the surrounding words), and q_v is the visual context (i.e., the image accompanying the sentence containing the mention), and a knowledge base $\mathcal{E} = \{e^i\}_{i=1}^K$ of K entities. Each entity $e^i = \{e_n^i, e_d^i, e_v^i\}$ consists of a name e_n^i , a textual description e_d^i , and a visual context e_v^i . The goal of MEL is to link the query q to the correct entity $e^* \in \mathcal{E}$.

The memory footprint of an entity in the knowledge base is defined as: *Entity Size (bytes)* = $N_f \times d \times 2$, where N_f is the number of features per entity, d is the dimensionality of each feature, and we assume float16 storage (2 bytes per value). Although additional compression techniques (quantization, pruning, etc.) could be used, this paper focuses on architectural choices that reduce N_f and d . The total KB index size is then obtained by multiplying this per-entity size by the number of entities.

In Table 1, we compare 13 state-of-the-art MEL models through the lens of the storage–computation–accuracy trade-off. We propose the following classification criteria: *High accuracy* refers to models that rank within the top quartile in terms of average H@1 performance on three widely used MEL datasets (namely, WikiDiverse [18], RichpediaMEL [17], and WikiMEL [17]). *Storage efficiency* refers to models that do not require storing representations for all textual tokens and image patches, thereby enabling a smaller KB index size. *Computational efficiency* refers to non-generative approaches (i.e., models not based on LLMs or MLLMs), as generative frameworks typically demand substantially more GPU memory due to their larger parameter counts and incur additional computational overhead and latency from autoregressive generation. Empirical evidence supporting this claim is provided in Section 4.

Some observations from the table follow. MIMIC [8], MELOV [15], M3EL [1], MMoE [2], FissFuse [7], and OTMEL [20] are computationally efficient, but sacrifice accuracy and storage efficiency. KGMEl [3], MMoE+DME [2], KGMEl+RR [3], GEMEL [14], FissFuse+KAR [7], I2CR [5], and UniMEL [4] all depend on LLMs or

MLLMs. They are therefore not computationally efficient, although some are storage-efficient. Moreover, KGMEl, MMoE+DME, KGMEl+RR use OpenAI GPT-3.5-turbo or GPT-4o-mini and thus incur API dependence and cost.

Table 1 highlights the trade-offs in MEL system design: models that achieve high accuracy tend to use large generative models, while those emphasizing storage compactness often sacrifice performance and storage efficiency. Notably, no existing model meets all three requirements simultaneously, motivating the need for an approach that accounts for all of them.

3 FAST-MEL: Fast, Accurate and STorage Efficient MEL

In this section, we describe our method for satisfying all three requirements in three stages: (1) *feature encoding*, which extracts token- and patch-level representations for queries and entities; (2) *feature pooling*, where we introduce a novel strategy that aggregates all features to reduce storage overhead; and (3) *contrastive training*, which incorporates a new hard-negative strategy to learn more discriminative representations and improve performance.

3.1 Feature Encoding

Following prior work [1, 2, 7, 8, 15, 20], we first extract textual and visual features for both queries and entities using pre-trained encoders. The textual and visual encoders are denoted by E_t and E_v respectively. For text encoding, the following templates are used:

$$\begin{aligned} F_t^q &= E_t([\text{CLS}] \ q_m \ [\text{EOT}] \ q_t \ [\text{EOT}]), \\ F_t^e &= E_t([\text{CLS}] \ e_n \ [\text{EOT}] \ e_d \ [\text{EOT}]). \end{aligned} \quad (1)$$

Here, F_t^q and $F_t^e \in \mathbb{R}^{S \times d_t}$ are, respectively, the query and entity token-level embeddings of size d_t . S denotes the length of the token sequence. Similarly, visual patch-level representations are obtained by passing the images through the visual encoder:

$$F_v^q = E_v(q_v), \quad F_v^e = E_v(e_v), \quad (2)$$

where F_v^q and $F_v^e \in \mathbb{R}^{P \times d_v}$ are the patch-level d_v -dimensional embeddings of the query and entity images. P is the number of image patches, including a [CLS] patch at the beginning of the sequence.

3.2 Feature Pooling

To leverage both token- and patch-level information while keeping representations compact, we propose a simple yet effective pooling strategy. This new approach combines all information into a single vector per query and entity, yielding a compact embedding that preserves task-relevant information. Importantly, it avoids storing feature vectors for all S textual tokens and P image patches (typically 40 tokens per text and 50 patches per image). Since the strategy is identical for entities and queries, we denote their textual and visual features as F_t and F_v , respectively. First, all features are projected into a shared embedding space of dimension d using learned linear projections:

$$\hat{F}_t = F_t W_t + b_t \in \mathbb{R}^{S \times d}, \quad \hat{F}_v = F_v W_v + b_v \in \mathbb{R}^{P \times d}, \quad (3)$$

where $W_t \in \mathbb{R}^{d_t \times d}$, $W_v \in \mathbb{R}^{d_v \times d}$, $b_t, b_v \in \mathbb{R}^d$ are learnable parameters. After projection, the textual and visual features can be

¹Code available at: <https://github.com/to2002td-cpu/FASTMEL.git>

Table 1: Comparison of 13 MEL Models under the Storage–compute–accuracy Trade-off. ([†]No vector index is stored)

Requirement	MIMIC	OTMEL	MELOV	M3EL	FissFuse	MMoE	KGMEI	GEMEL [†]	MMoE+DME	FissFuse+KAR	KGMEI+RR	I2CR [†]	UniMEL	FAST-MEL
High accuracy	×	×	×	×	×	×	×	×	×	×	✓	✓	✓	✓
Comput. efficiency	✓	✓	✓	✓	✓	✓	×	×	×	×	×	×	×	✓
Storage efficiency	×	×	×	×	×	×	✓	–	×	×	✓	–	✓	✓

described as a set F of length $L = S + P$ of feature vectors:

$$F = \{f_i \mid f_i \in \hat{F}_t \text{ or } f_i \in \hat{F}_e\}, \quad |F| = L. \quad (4)$$

Given this set, our goal is to retain only the most useful information. To achieve this, we propose learning a weighted average of the features. Each vector in the set F is therefore assigned a score using a two-layer MLP with a nonlinear $\tanh$ activation ϕ :

$$s_i = W_2^\top \phi(W_1 f_i + b_1) + b_2, \quad i = 1, \dots, L, \quad (5)$$

where $W_1 \in \mathbb{R}^{h \times d}$, $W_2 \in \mathbb{R}^h$, $b_1 \in \mathbb{R}^h$, and $b_2 \in \mathbb{R}$ are learnable parameters, and h is the hidden size of the MLP. The resulting scalar scores are normalized using a softmax function to obtain non-negative coefficients that sum to one. The final pooled vector $\tilde{F} \in \mathbb{R}^d$ is computed as the weighted sum of the features:

$$\tilde{F} = \sum_{i=1}^L \alpha_i f_i, \quad \alpha_i = \frac{\exp(s_i)}{\sum_{j=1}^L \exp(s_j)}. \quad (6)$$

This procedure compresses both textual token-level and visual patch-level information of an entity or a query into a single fixed-dimensional vector. It also allows the model to weight each token and patch embeddings separately. At indexing time, only this pooled feature needs to be stored.

3.3 Contrastive Training with Hard Negatives

FAST-MEL is trained to learn dense representations for queries and entities using a contrastive learning framework, which encourages the model to assign higher similarity scores to correct query-entity pairs while pushing apart incorrect ones. Let $\tilde{F}^q$ denotes the pooled representation of a query, and $\tilde{F}^e$ the representation of an entity. The matching score between a query q and an entity $e \in \mathcal{E}$ is computed using cosine similarity between their pooled representations:

$$s(q, e) = \cos(\tilde{F}^q, \tilde{F}^e) = \frac{(\tilde{F}^q)^\top \tilde{F}^e}{\|\tilde{F}^q\| \|\tilde{F}^e\|}. \quad (7)$$

Training is performed in mini-batches, where for each query, all non-matching entities in the batch serve as *in-batch negatives*. To further improve discriminability, we introduce *hard negative* entities for each query. These hard negatives are mined offline from the KB comparing each query to all entities using BM25 [13], a classical sparse retrieval method. This incurs minimal preprocessing overhead prior to training (under 30 seconds on a standard CPU for 15k+ examples). The query-entity contrastive loss is defined as a cross-entropy [16] over the batch, including both in-batch and hard negatives, with a learnable temperature τ :

$$\mathcal{L} = - \sum_{q \in B} \log \frac{\exp(\tau s(q, e_q^*))}{\sum_{e \in (E \cup H_q) \setminus \{e_q^*\}} \exp(\tau s(q, e))}, \quad (8)$$

where e_q^* denotes the correct entity for query q , H_q is the set of hard negatives for q , and E is the set of entities in the batch. During training, we randomly sample one hard negative from the top 10

most similar entities for each query at each step. Minimizing this loss encourages the model to bring correct query-entity pairs closer in the embedding space while pushing apart both in-batch and hard negative entities, resulting in more discriminative representations.

4 Experiments

FAST-MEL is evaluated on three public MEL datasets: WikiDiverse, RichpediaMEL, and WikiMEL. For fair comparison with prior work, we use the same dataset splits and the same subsets of Wikidata for our KB. We report standard evaluation metrics, including Hits@1 (H@1) and Mean Reciprocal Rank (MRR), to assess entity linking performance. Our model is trained end-to-end with a batch size of 64 and a learning rate of 1×10^{-5} , selected via a grid search over batch sizes $\{16, 32, 64, 128\}$ and learning rates $\{1 \times 10^{-4}, 1 \times 10^{-5}, 1 \times 10^{-6}\}$. Regarding training time, FAST-MEL converges in 20–30 minutes on a single A100 40GB GPU. Results for our model are reported as the average over three runs, including the standard deviation. FAST-MEL is compared against two types of baselines: (1) encoder-based methods [1, 2, 7, 8, 15, 20], which are fast but require a large KB index size, and (2) LLM/MLLM-based approaches [2–5, 7, 14], which are slower but often achieve higher accuracy. Note that DME, KAR, and RR refer to LLM-based modules added on top of encoder-based methods to improve accuracy.

Table 2 presents the performance of our method and the baseline models across the three benchmarks. All baseline results are directly reported from their respective original publications. FAST-MEL achieves an average H@1 score of 86.08, outperforming all encoder-based methods by a margin of at least three points. In addition, it is the most storage-efficient, with an index size close to $17\times$ smaller than encoder-based approaches, and on par with the most storage-economical solution, KGMEI. Compared with LLM/MLLM-based approaches, FAST-MEL surpasses the majority of them (4 out of 7). The three methods that achieve higher performance—KGMEI+RR, I2CR, and UniMEL—incur substantially greater computational costs and latency, which we discuss below.

Table 3 compares the online inference time of FAST-MEL with the strongest encoder-based baseline (MMoE) and the best LLM/MLLM-based approach (UniMEL). Since all encoder-based baselines share the same CLIP backbone, MMoE is representative of their inference regime. Similarly, UniMEL is representative of LLM/MLLM-based approaches, which all rely on large generative models and autoregressive decoding, observation further supported by I2CR, which reports inference times similar to UniMEL [5]. The experiment is conducted on 1,000 queries and 1,000 entities sampled from WikiMEL, using a batch size of 16 on a single A100 40GB GPU. We report the average time required to retrieve the most probable KB entity for each query. FAST-MEL achieves the lowest end-to-end matching time (1.26 ms), making it orders of magnitude faster than UniMEL (1.46 s) while also outperforming the best encoder-based

Table 2: Comparison of MEL Models on WikiDiverse, RichPedia-MEL, and WikiMEL. (All baseline values are taken from the original publications; ‘-’ Value not reported; † Models using GPT-3.5-turbo; * Models using GPT-4o-mini)

Model	Venue	#Params	Entity Size (bytes)↓	WikiDiverse H@1↑	WikiDiverse MRR↑	RichPedia-MEL H@1↑	RichPedia-MEL MRR↑	WikiMEL H@1↑	WikiMEL MRR↑	Avg H@1↑ (Δ/FAST-MEL)
Encoder-Only (Not Storage Efficient)										
MIMIC	KDD	150M	17 280	63.51	73.44	81.02	86.95	87.98	91.82	77.50 (-8.58)
OTMEL	ACL	150M	17 280	66.07	76.57	83.30	88.80	88.97	92.32	79.45 (-6.63)
MELOV	ACL	150M	17 280	67.32	81.29	84.14	88.26	88.91	92.30	80.12 (-5.96)
M3EL	KDD	150M	17 280	70.36	81.72	82.82	89.04	88.84	92.53	80.67 (-5.41)
FissFuse	ACM MM	150M	19 200	80.30	88.11	–	–	84.80	90.26	82.55 (-3.53)
MMoE	KDD	150M	17 280	74.59	75.43	84.36	88.27	89.07	92.59	82.67 (-3.41)
Encoder-Only (Pooled)										
FAST-MEL		150M	1024	84.48±0.06	89.18±0.09	84.30±0.73	88.12±0.53	89.46±0.21	92.78±0.12	86.08
Using LLM/MLLM (Slow Inference)										
KGMEL	SIGIR	*	1024	82.12	86.00	76.40	80.94	87.29	89.99	81.94 (-4.14)
GEMEL	COLING	7B	–	86.30	–	–	–	82.60	–	84.45 (-1.63)
MMoE+DME	KDD	†	17 280	77.57	84.23	85.51	89.86	90.77	93.75	84.62 (-1.46)
FissFuse+KAR	ACM MM	7B	19 200	83.29	89.81	–	–	87.89	92.02	85.59 (-0.49)
KGMEL+RR	SIGIR	†*	1024	88.23	90.84	85.21	88.08	90.58	93.04	88.01 (+1.93)
I2CR	ACM MM	8B	–	92.60	–	94.70	–	86.80	–	91.37 (+5.29)
UniMEL	CIKM	15B	8192	92.90	–	94.80	–	94.10	–	93.93 (+7.85)

Table 3: Average Online Inference Time per Query on WikiMEL. (‘-’ indicates that a step is not applicable to the corresponding model; ‘Aug.’ stands for augmentation)

Step	UniMEL	MMoE	FAST-MEL
Query aug. with image (LLaVa) (ms)	1122.92	–	–
Query textual aug. (LLaMa) (ms)	85.33	–	–
Embedding creation (ms)	81.14	1.14	1.11
Top-K retrieval (ms)	49.52	–	–
Final scoring (ms)	119.74	4.16	0.15
Avg total matching time per query (ms)	1458.65	5.30	1.26

Table 4: Ablation Study

Setting	RichP H@1	WikiD H@1	WMEL H@1	Avg H@1
Pooling Strategy				
CLS-Mean	8.71	20.66	26.29	18.55
CLS-MLP	9.81	22.62	27.25	19.89
Mean Pooling	83.73	81.07	88.12	84.31
Max Pooling	80.46	77.90	85.55	81.30
Modality				
Text only	81.05	85.80	87.29	84.71
Hard Negative Mining Strategy				
None	68.86	72.96	82.04	74.62
SBERT	82.05	85.10	89.40	85.52
FAST-MEL	84.48±0.06	84.30±0.73	89.46±0.21	86.08

model. We also evaluated the other most storage-efficient method, KGMEL, in the same setup (results not shown in the table), and found that its LLM-based generation of synthetic RDF triples for each query caused a substantial overhead, exceeding 4 seconds per query. This highlights the critical bottleneck of these generative methods compared to encoder-based approaches.

To further analyze our design choices, we conducted an ablation study (Table 4). First, we investigated different pooling strategies. *CLS-Mean* averages the CLS tokens from the image and text encoders (Eq. 1 and Eq.2). *CLS-MLP* keeps only the CLS tokens in F

(Eq. 4) and applies our learned pooling (Eq. 6). In both cases, severe performance degradation is observed, indicating that relying solely on CLS representations fails to capture task-relevant information. In contrast, classical pooling of all token- and patch-level features using *mean* or *max* operators yields competitive performance. While these approaches highlight the importance of preserving token- and patch-level information, they consistently perform worse than our learned pooling method. Similarly to previous studies [3, 4, 7], an image ablation was conducted to evaluate the contribution of visual features. To do so, image tokens were removed from F (Eq. 4). This ablation resulted in a slight performance improvement on WikiDiverse, suggesting that images possibly introduce more noise than valuable information for our model on this benchmark. Nevertheless, the full multimodal setup remains superior overall, achieving the highest average H@1 score across datasets. Regarding hard negative mining, our ablation shows that it is crucial for learning a discriminative embedding space. Removing this component reduces performance to 74.62% Avg H@1. Trying to replace BM25 with an SBERT [12] model does not improve performance but introduces a significantly higher computational cost. Finally, the sensitivity of the model to the embedding dimension d was analyzed (cf. Figure 2). H@1 improves rapidly from $d = 32$ to $d = 256$. An optimal peak is observed at $d = 512$ (i.e., an entity size of 1024 bytes), providing a good balance between representational capacity and storage efficiency. Increasing the dimension to 1024 results in a slight decrease in performance.

5 Conclusion

In this article, we analyzed MEL from a new perspective: the trade-off between storage efficiency, computational efficiency, and high accuracy, which is essential for use in large-scale practical settings. Highlighting that state-of-the-art MEL systems do not meet this triple objective, we introduced FAST-MEL, a lightweight encoder-based solution that satisfies these three key requirements, relying on compact representations and hard negative mining. Extensive

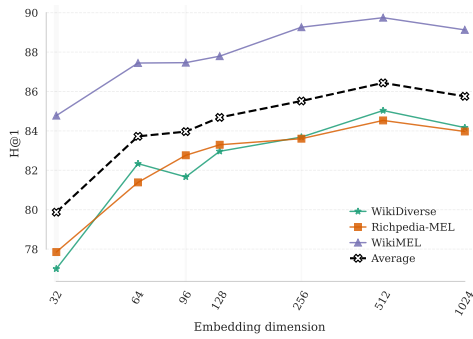


Figure 2: Impact of the Final Embedding Dimension d on $H@1$ Performance.

experiments demonstrate that FAST-MEL achieves strong performance without compromising efficiency.

Acknowledgments

This work was in part publicly funded through the French ANR (Agence Nationale de la Recherche) under the project AGAPE with the reference ANR-24-CE38-7253.

References

- [1] Zhiwei Hu, Víctor Gutiérrez-Basulto, Ru Li, and Jeff Z. Pan. 2025. Multi-level Matching Network for Multimodal Entity Linking. In *Proceedings of the 31st ACM SIGKDD conference on Knowledge Discovery and Data Mining (KDD'25)*. Association for Computing Machinery, New York, NY, 508–519. doi:10.1145/3690624.3709306
- [2] Zhiwei Hu, Víctor Gutiérrez-Basulto, Zhiliang Xiang, Ru Li, and Jeff Z. Pan. 2025. Multi-level Mixture of Experts for Multimodal Entity Linking. In *Proceedings of the 31st ACM SIGKDD conference on Knowledge Discovery and Data Mining (KDD'25)*, Vol. 2. Association for Computing Machinery, New York, NY, 979–990. doi:10.1145/3711896.3737060
- [3] Juyeon Kim, Geon Lee, Taeuk Kim, and Kijung Shin. 2025. KGME: Knowledge Graph-enhanced Multimodal Entity Linking. In *Proceedings of the 48th international ACM SIGIR conference on Research and Development in Information Retrieval (SIGIR 2025)*. Association for Computing Machinery, New York, NY, 3015–3019. doi:10.1145/3726302.3730217
- [4] Qi Liu, Yongyi He, Tong Xu, Defu Lian, Che Liu, Zhi Zheng, and Enhong Chen. 2024. UniMEL: A Unified Framework for Multimodal Entity Linking with Large Language Models. In *Proceedings of the 33rd ACM international Conference on Information and Knowledge Management (CIKM'24)*. Association for Computing Machinery, New York, NY, 1909–1919. doi:10.1145/3627673.3679793
- [5] Ziyang Liu, Junwen Li, Kaiwen Li, Tong Ruan, Chao Wang, Xinyan He, Zongyu Wang, Xuezhi Cao, and Jingping Liu. 2025. I2CR: Intra- and Inter-modal Collaborative Reflections for Multimodal Entity Linking. In *Proceedings of the 33rd ACM international conference on Multimedia (MM'25)*. Association for Computing Machinery, New York, NY, 4942–4951. doi:10.1145/3746027.3755674
- [6] Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. Entity-based Knowledge Conflicts in Question Answering. In *Proceedings of the 2021 conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*. Association for Computational Linguistics, 7052–7063. doi:10.18653/v1/2021.emnlp-main.565
- [7] Pengfei Luo, Tong Xu, Che Liu, Suojuan Zhang, Linli Xu, Minglei Li, and Enhong Chen. 2024. Bridging Gaps in Content and Knowledge for Multimodal Entity Linking. In *Proceedings of the 32nd ACM international conference on Multimedia (MM'24)*. Association for Computing Machinery, New York, NY, 9311–9320. doi:10.1145/3664647.3681661
- [8] Pengfei Luo, Tong Xu, Shiwei Wu, Chen Zhu, Linli Xu, and Enhong Chen. 2023. Multi-grained Multimodal Interaction Network for Entity Linking. In *Proceedings of the 29th ACM SIGKDD conference on Knowledge Discovery and Data Mining (KDD'23)*. Association for Computing Machinery, New York, NY, 1583–1594. doi:10.1145/3580305.3599439
- [9] Edgar Meij, Krisztian Balog, and Daan Odijk. 2014. Entity Linking and Retrieval for Semantic Search. In *Proceedings of the 7th ACM international conference on Web Search and Data Mining (WSDM'14)*. Association for Computing Machinery, New York, NY, 683–684. doi:10.1145/2556195.2556201
- [10] Seungwhan Moon, Leonardo Neves, and Vitor Carvalho. 2018. Multimodal Named Entity Disambiguation for Noisy Social Media Posts. In *Proceedings of the 56th annual meeting of the Association for Computational Linguistics (ACL 2018)*. Association for Computational Linguistics, 2000–2008. doi:10.18653/v1/P18-1186
- [11] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models from Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, Vol. 139. PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a/radford21a.pdf>
- [12] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP 2019)*. Association for Computational Linguistics, 3982–3992. doi:10.18653/v1/D19-1410
- [13] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. 1994. Okapi at TREC-3. In *Proceedings of the 3rd Text Retrieval Conference (TREC-3)*. Department of Commerce, NIST, 109–126. <https://trec.nist.gov/pubs/trec3/papers/city.ps.gz>
- [14] Senbao Shi, Zhenran Xu, Baotian Hu, and Min Zhang. 2024. Generative Multimodal Entity Linking. In *Proceedings of the 2024 joint international conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. ELRA and ICCL, 7654–7665. <https://aclanthology.org/2024.lrec-main.676/>
- [15] Xuhui Sui, Ying Zhang, Yu Zhao, Kehui Song, Baohang Zhou, and Xiaojie Yuan. 2024. MELOV: Multimodal Entity Linking with Optimized Visual Features in Latent Space. In *Proceedings of Findings of the Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics, 816–826. doi:10.18653/v1/2024.findings-acl.46
- [16] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. *Representation Learning with Contrastive Predictive Coding*. arXiv:1807.03748 doi:10.48550/arXiv.1807.03748
- [17] Peng Wang, Jiangheng Wu, and Xiaohang Chen. 2022. Multimodal Entity Linking with Gated Hierarchical Fusion and Contrastive Training. In *Proceedings of the 45th international ACM SIGIR conference on Research and Development in Information Retrieval (SIGIR 2022)*. Association for Computing Machinery, New York, NY, 938–948. doi:10.1145/3477495.3531867
- [18] Xuwu Wang, Junfeng Tian, Min Gui, Zhixu Li, Rui Wang, Ming Yan, Lihan Chen, and Yanghua Xiao. 2022. WikiDiverse: A Multimodal Entity Linking Dataset with Diversified Contextual Topics and Entity Types. In *Proceedings of the 60th annual meeting of the Association for Computational Linguistics (ACL 2022)*. Association for Computational Linguistics, 4785–4797. doi:10.18653/v1/2022.acl-long.328
- [19] Wenhan Xiong, Mo Yu, Shiyu Chang, Xiaoxiao Guo, and William Yang Wang. 2019. Improving Question Answering over Incomplete KBs with Knowledge-Aware Reader. In *Proceedings of the 57th annual meeting of the Association for Computational Linguistics (ACL 2019)*. Association for Computational Linguistics, 4258–4264. doi:10.18653/v1/P19-1417
- [20] Zefeng Zhang, Jiawei Sheng, Chuang Zhang, Liangyunzhi Liangyunzhi, Wenyan Zhang, Siqi Wang, and Tingwen Liu. 2024. Optimal Transport Guided Correlation Assignment for Multimodal Entity Linking. In *Proceedings of Findings of the Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics, 4103–4117. doi:10.18653/v1/2024.findings-acl.243